The Implicit Fallback (`src/cloth_implicit.py`)

1 Why a Second Solver?

The MPM simulator is accurate but uses a tiny timestep (1e-4 s) to stay stable (explicit symplectic Euler [paper2, §3.5]). The neural network in M3 will be fast but imprecise — it will occasionally produce wrong predictions.

The implicit fallback is the **safety net**: when the detector fires, we fall back to a solver that is:

- **Unconditionally stable** — can take large timesteps (h = 0.02 s, 200× larger than MPM) without blowing up. This is the core argument of [paper13]: implicit Euler's stability is independent of stiffness.
- **Simple** — easy to run on the same particle positions the MPM and neural solvers maintain.
- **Cheap** — cheaper than a full MPM step at large timesteps.

The Baraff–Witkin (1998) implicit Euler mass-spring solver [paper13] fits all three criteria. [paper7] uses full MPM as its fallback; we use the implicit solver as a cheaper alternative for cloth, where the mass-spring model is more natural than an MPM grid solver.

2 Spring Topology

The 64×64 cloth particles are connected by three families of springs, built once in `build_topology()`. These correspond directly to the three *condition* families in [paper13, §2–3]: stretch (C_stretch), shear (C_shear), and bend (C_bend), adapted from the original continuum-on-triangles formulation to a regular quad grid with explicit springs.

Family	**Count (64×64 grid)**	**Stiffness**	**[paper13] analog**
Structural	8064	`E × thickness`	Stretch condition C_stretch
Shear	7938	`0.5 × structural`	Shear condition C_shear
Bend	7744	`0.1 × structural`	Bend condition C_bend (dihedral angle)

[paper13] notes that anisotropic bend stiffness `(k_u Du² + k_v Dv²) / (Du² + Dv²)` can influence the warp/weft directions differently. We use an isotropic simplification (equal stiffness in all directions) since our cloth is isotropic at M2 scope.

All stiffnesses derive from the same YAML `young_modulus_warp_pa`, so the implicit and MPM solvers share material parameters.

3 Spring Force and Jacobian

For each spring connecting particles `i` and `j` [paper13, Eq. 7–8]:

e = x_j - x_i (current edge vector)

L = |e| (current length)

e_hat = e / L (unit direction)

force on i = +k * (L - L_0) * e_hat (Hooke's law; toward j when stretched)

force on j = -force on i

The Jacobian of this force (needed for implicit Euler) is:

$\partial f_i/\partial x_j = k * [(L-L_0)/L * (I - e\_hat\ e\_hat^T) + e\_hat\ e\_hat^T]$

The first term is the *transverse* stiffness (perpendicular to the spring), scaled down by `(L-L₀)/L` — when the spring is at rest length this term vanishes. The second term is the *axial* stiffness (along the spring direction) — always present. This matches [paper13, Eq. 7–8].

Damping follows [paper13, Eq. 11]: `d = -k_d (∂C/∂x) Ċ(x)` — derived from the same condition as the elastic force so that damping is aligned with the spring modes and does not penalise rigid-body motion.

4 The Implicit Euler System

Explicit Euler is: `v_new = v_old + dt * f(x_old) / m`. It's unstable for stiff springs (requires tiny dt).

Implicit Euler instead evaluates forces at the *new* state. This is unconditionally stable but requires solving a linear system each step. Linearising `f(x_new)` and substituting (following [paper13, §4, Eq. 6 and symmetric form Eq. 15]):

$A\,\Delta v = b$

$A = M - h\cdot\partial f/\partial v - h^2\cdot\partial f/\partial x$

$b = h\cdot(f_0 + h\cdot\partial f/\partial x\cdot v\_old)$

`M` is the diagonal mass matrix. `h = dt`. `A` is sparse and symmetric positive-definite (with the symmetry trick from [paper13, §5.1]).

[paper13] uses h = 0.02 s for human-motion cloth simulations; our config also uses dt = 0.02 s for the implicit solver, matching their recommended operating point.

5 Modified PCG Solver

`_modified_pcg` solves `A Δv = b` iteratively using Preconditioned Conjugate Gradient. The preconditioner is the diagonal of `A` (Jacobi): `P_ii = 1/A_ii` [paper13, §5.3].

The key modification is the **filter** `S`, taken verbatim from [paper13, §5.3]. For pinned particles, `S_i = 0` (zero out that particle's DOFs). This is applied at every CG step by zeroing the pinned entries in the iterate and residual:

```
def _filter(self, w):
    out = w.copy()
    if self.pinned.any():
        out[self.pinned] = 0.0
    return out
```

[paper13]'s key claim: *"A property of our approach, however, is that the constraints are maintained exactly, regardless of the number of iterations taken by the linear solver."*. Because the filter is applied *at every iteration*, pinned constraints are **exact** regardless of how many CG iterations run — even one iteration. Verified by `test_pinned_corners_exact`.

[paper13] reports 50–100 CG iterations for a 6,000-node system. Our 4,096-node cloth (smaller) converges in < 50 iterations, well within budget.

6 Why the Fallback Doesn't Handle Sphere Collision

`ImplicitClothSim.step()` applies only gravity + spring forces. It has no sphere or ground collision.

This is by design, following the hybrid pipeline pattern from [paper7]. In that paper, the fallback (full MPM step) handles its own collision. In our adaptation, the fallback is invoked mid-rollout with particle state already resolved by MPM; it takes only one or a few steps before handing control back to the neural solver, so the cloth doesn't drift into the sphere within the fallback window.